

School of Computer Science and Engineering

Fall Semester 2024-25

CAT I Key

SLOT: B1 Slot

Programme Name & Branch: B. Tech CSE

Course Name & Code: Operating Systems & BCSE303L

Class Number (s):

Faculty Name (s):

Exam Duration: 90 Min.

Maximum Marks: 50

Q. No.	Question	Max Marks	CO	BL
1.	A <i>multiprocessor</i> system contains <i>more than one</i> CPU (also known as processor) and they work in parallel. This is called Simultaneous Multiprocessing (SMP). Multiprocessor systems have a special type of motherboard which has several CPU sockets. A <i>multicore</i> system contains more than one execution core on <i>one</i> CPU. The exact meaning of multicore depends on the architecture, but essentially a certain subset of the CPU's components is replicated, so that several <i>cores</i> can work in parallel on isolated CPU operations. This is called Chip-level Multiprocessing (CMP). A multicore chip can have a dedicated execution unit and L1 cache for a core and a shared L2 cache for the entire CPU. Multiprocessor means having several CPUs whereas multicore means having several cores. The operating system controls all a computer system's hardware resources, including processor, memory, storage space, network controller, peripherals, and user input/output. As such, it controls many resources that need protection. Some of the tools an OS often utilizes include user identity management, anti-malware software, network firewalls, and encryption. The operating system operates at many levels of security. It controls access to hard drives, network controllers, and user interfaces. Therefore, it participates in security at the data, user, and network levels. It also plays a role in mitigating damage at the physical security level through the use of backup and restore operations.	5+5	CO 1	BL 2

2.	Definition - A system call is a request from computer software to an operating system's kernel. (1 mark) Need - System calls allow user-level processes to request services of the operating system Simplicity – No need to write complex code (1 mark) Parameter passing methods (3 marks) 1) Parameters should be pushed on or popped off the stack by the operating system. 2) Parameters can be passed in registers. 3) When there are more parameters than registers, it should be stored in a block, and the block address should be passed as a parameter to a register. Instruction Mode (5 marks) <table><tr><th>S.No</th><th>Instruction</th><th>Privileged or Non-Privileged Instruction</th><th>Mode (Kernel mode or User Mode)</th></tr><tr><td>i)</td><td>Reading system time</td><td>Non-Privileged</td><td>User Mode</td></tr><tr><td>ii)</td><td>Clear Memory</td><td>Privileged</td><td>Kernel Mode</td></tr><tr><td>iii)</td><td>Opening and reading a file</td><td>Privileged</td><td>Kernel Mode</td></tr><tr><td>iv)</td><td>Set the timer</td><td>Privileged</td><td>Kernel Mode</td></tr><tr><td>v)</td><td>Performing arithmetic operation</td><td>Non-Privileged</td><td>User Mode</td></tr></table>	S.No	Instruction	Privileged or Non-Privileged Instruction	Mode (Kernel mode or User Mode)	i)	Reading system time	Non-Privileged	User Mode	ii)	Clear Memory	Privileged	Kernel Mode	iii)	Opening and reading a file	Privileged	Kernel Mode	iv)	Set the timer	Privileged	Kernel Mode	v)	Performing arithmetic operation	Non-Privileged	User Mode	5+5	CO 1	BL 4
S.No	Instruction	Privileged or Non-Privileged Instruction	Mode (Kernel mode or User Mode)																									
i)	Reading system time	Non-Privileged	User Mode																									
ii)	Clear Memory	Privileged	Kernel Mode																									
iii)	Opening and reading a file	Privileged	Kernel Mode																									
iv)	Set the timer	Privileged	Kernel Mode																									
v)	Performing arithmetic operation	Non-Privileged	User Mode																									
3.	<table><tr><th>User Level Thread</th><th>Kernel Level Thread</th></tr><tr><td>User-level threads are faster to create and manage.</td><td>Kernel level threads are slower to create and manage.</td></tr><tr><td>Implemented by a thread library user level.</td><td>Operating system support directly to Kernel threads.</td></tr><tr><td>User level thread can run on any operating system.</td><td>Kernel level threads are specific to the operating system.</td></tr><tr><td>Support provided at the user level called user-level thread</td><td>Support may be provided by Kernel is called Kernel level threads.</td></tr><tr><td>Multithread applications cannot take advantage of multiprocessing.</td><td>Kernel routines themselves can be multithreaded.</td></tr><tr><td>Implementation of User threads is easy.</td><td>Implementation of Kernel thread is complicated.</td></tr><tr><td>Context switch time is less.</td><td>Context switch time is more.</td></tr><tr><td>Context switch requires no hardware support.</td><td>Hardware support is needed.</td></tr><tr><td>Example: User-thread libraries include POSIX Pthreads, Mach C-threads, and Solaris 2 UI-threads.</td><td>Example: Windows NT, Windows 2000, Solaris 2, BeOS, and Tru64 UNIX (formerly Digital UNIX)-support kernel threads.</td></tr></table>	User Level Thread	Kernel Level Thread	User-level threads are faster to create and manage.	Kernel level threads are slower to create and manage.	Implemented by a thread library user level.	Operating system support directly to Kernel threads.	User level thread can run on any operating system.	Kernel level threads are specific to the operating system.	Support provided at the user level called user-level thread	Support may be provided by Kernel is called Kernel level threads.	Multithread applications cannot take advantage of multiprocessing.	Kernel routines themselves can be multithreaded.	Implementation of User threads is easy.	Implementation of Kernel thread is complicated.	Context switch time is less.	Context switch time is more.	Context switch requires no hardware support.	Hardware support is needed.	Example: User-thread libraries include POSIX Pthreads, Mach C-threads, and Solaris 2 UI-threads.	Example: Windows NT, Windows 2000, Solaris 2, BeOS, and Tru64 UNIX (formerly Digital UNIX)-support kernel threads.	10	CO 1	BL 5				
User Level Thread	Kernel Level Thread																											
User-level threads are faster to create and manage.	Kernel level threads are slower to create and manage.																											
Implemented by a thread library user level.	Operating system support directly to Kernel threads.																											
User level thread can run on any operating system.	Kernel level threads are specific to the operating system.																											
Support provided at the user level called user-level thread	Support may be provided by Kernel is called Kernel level threads.																											
Multithread applications cannot take advantage of multiprocessing.	Kernel routines themselves can be multithreaded.																											
Implementation of User threads is easy.	Implementation of Kernel thread is complicated.																											
Context switch time is less.	Context switch time is more.																											
Context switch requires no hardware support.	Hardware support is needed.																											
Example: User-thread libraries include POSIX Pthreads, Mach C-threads, and Solaris 2 UI-threads.	Example: Windows NT, Windows 2000, Solaris 2, BeOS, and Tru64 UNIX (formerly Digital UNIX)-support kernel threads.																											

4.

Round Robin (Time Slice=3 ms): (4 Marks)

Gantt Chart:



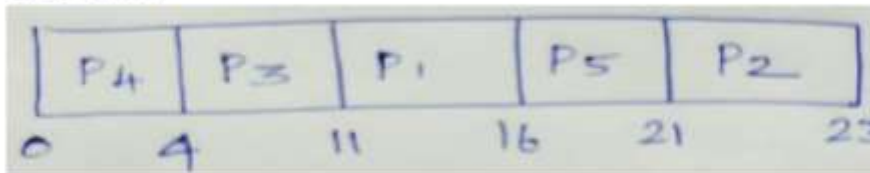
Process	Waiting Time	Turnaround Time
P1	12	17
P2	6	8
P3	13	20
P4	6	10
P5	12	17

Average waiting time: 9.8 ms

Average Turnaround time: 14.4 ms

Priority (Non preemptive) : (3 Marks)

Gantt Chart:



Process	Waiting Time	Turnaround Time
P1	6	11
P2	17	19
P3	1	8
P4	0	4
P5	13	18

Average waiting time: 7.4 ms

Average Turnaround time: 12 ms

10

CO
2BL
3

5.

Calculation of CPU Burst Time: (2 Marks)

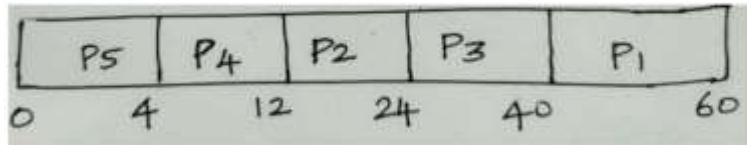
Process	Total Execution Time (ms)	CPU Burst Time [80% of total execution time] (ms)
P1	25	20
P2	15	12
P3	20	16
P4	10	8
P5	5	4

10

CO
2BL
3

SIF: (4 Marks)

Gantt Chart:



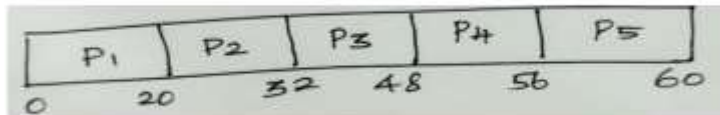
Process	Waiting Time	Turnaround Time
P1	40	60
P2	12	24
P3	24	40
P4	4	12
P5	0	4

Average waiting time: 16 ms

Average Turnaround time: 28 ms

FCFS: (4 Marks)

Gantt Chart:



Process	Waiting Time	Turnaround Time
P1	0	20
P2	20	32
P3	32	48
P4	48	56
P5	56	60

Average waiting time: 31.2 ms

Average Turnaround time: 43.2 ms